

Lecture 1: Randomness, References, Pointers

ICS 46

Data Structure Implementation and Analysis

09/29/26

Tom Yeh, PhD (he/him)
Assistant Professor of Teaching
Computer Science Department
UC Irvine



Cloudflare's Wall of Entropy

Class News

- Lab 1 released, due Mon 10/5
 - Environment set up
 - Small program to test compilation
 - Submit to Gradescope
 - For all hws, labs, projects
- Pensive link
- Interview questions traditionally focused on data structures and algorithms
- OH calendar updated

	Monday	Tuesday	Wednesday	Thursday	Friday
8:00 AM	ICS 458A (Raheeb)		ICS 458F(Yutong, start from 8:30)		
9:00 AM	ICS 458A (Raheeb)		ICS 458F(Yutong)	ICS 458F (Nazmul)	
10:00 AM	ICS 458A (Raheeb)	ICS 458B (Jun)		ICS 458F (Nazmul), ICS 458B (Jun)	
11:00 AM		ICS 458B (Jun, ends at 11:30)		ICS 458B (Jun, ends at 11:30)	
12:00 PM					
1:00 PM					TBD (Robin)
2:00 PM					TBD (Robin)
3:00 PM			TBD (Nazmul)		
4:00 PM		Lecture 3:30-4:50	ICS 458B (Robin)	Lecture 3:30-4:50	Discussion
5:00 PM		Prof. Yeh 5-6pm		Prof. Yeh 5-6pm	Discussion
6:00 PM			ICS 458FC(Yutong)		
Discussion					
OH w/ Prof Yeh, 5 - 6 pm, ICS1 195					
TA OH TBD					

Today's Goals

- Randomness
- References
- Pointers
- Smart Pointers

Programming Environment

- VS Code & C++
- Consistent environment for development
 - [UCI ICS Openlab JupyterHub](#)
 - Grading will be based on this environment
 - [Setup directions](#) provided in Lab 1
- Turn off copilot or any other AI assistance

Random

Randomness

Sequence of values is random if:

help_outline

Unpredictability

Cannot predict pattern better than by random chance

autorenew **Generative**

Goal

We want to be able to generate random sequences

Key Use Cases

psychology

Machine Learning

Weight initialization, data splitting, and stochastic optimizations

security

Security & Encryption

Random key generation, cryptographic tokens, and secure channels

science

Computer Simulation

Exploring probabilistic outcomes, seed variations, and statistical models

sports_esport

S

Games

World generation, procedural levels, NPC decisions, and gameplay balance

Generating Random Sequences

Random Number Generators (RNG)

grain_{Type 01}

1) Truly Random (TRNG) / Hardware (HRNG)

Key Characteristics:

- Closest practical approximation to true randomness
- Generated by measuring constantly changing physical phenomena

Examples & Sources of Entropy:

- Dice rolling, coin flipping, shuffling cards
- Network traffic, temperature, **lava lamps**
- Atmospheric noise (random.org)

Trade-off / Constraint:

Collecting entropy takes time ⇒ limited generation rate

memory_{Type 02}

2) Pseudo Random (PRNG)

Key Characteristics:

- Look random, but are in fact calculated deterministically
- Sufficient for most general software use cases

Determinism & Reproducibility

Given the same starting seed, PRNGs output the exact same sequence every time.

Primary Advantage:

Works at high speed of computing hardware

C++ Pseudorandom Generation

Seeding the Random Engine: Fixed Seed vs. Entropy Device

APPROACH 01

Fixed Seed (Reproducible)

Best For: Testing, simulations & debugging

To generate pseudorandom data, we create the generator with a seed. For simulations or testing, when we need reproducibility, we use a fixed seed such as 123 (unsigned integer).

```
#include <random>

const unsigned int SEED = 123;

std::default_random_engine engine{SEED};
```

Key Property: Produces identical sequence on every execution.

APPROACH 02

Entropy Source (Deployment)

Best For: Production & runtime randomness

For deployment, we can use a source of entropy to seed the random generator. Creates an object that lets us measure entropy. Once created, the device is called like a function to generate 1 random value.

```
#include <random>

std::random_device device;

std::default_random_engine engine{device()};
```

Key Property: Non-deterministic seed from hardware/OS.

C++ Pseudorandom Generation

Specifying Value Distributions: Uniform vs. Normal (Gaussian)

DISTRIBUTION 01

Uniform Integer

Generates integers between **min** and **max** (inclusive), with each possible value being equally likely.

C++ Code Example:

```
// Equal chance for 1, 2, 3, 4, 5, or 6
std::uniform_int_distribution<int>
distribution{1, 6};
```

Key Property:

Flat probability density across the entire specified range.

Uniform Distribution, min = 0, max= 10

```
0 ****
1 ****
2 ****
3 ****
4 ****
5 ****
6 ****
7 ****
8 ****
9 ****
10 ****
```

DISTRIBUTION 02

Normal / Gaussian

Produces a bell curve distribution centered at mean. Occurs frequently in nature and real-world data (e.g. exam scores).

C++ Code Example:

```
double mean = 10.0;
double stddev = 3.0;
std::normal_distribution<double>
d(mean, stddev);
```

Key Parameters:

Mean (center) & **StdDev** (standard deviation / spread).

Normal Distribution, mean = 5.0, stddev = 2.0

```
-2
-1
0
1 *
2 **
3 *****
4 *****
5 *****
6 *****
7 *****
8 ***
9 *
10
11
12
```

C++ Pseudorandom Generation

Simulating Dice Rolling with <random>

```
#include <random>
#include <iostream>

// 1. Source of Entropy & Engine
std::random_device device;
std::default_random_engine engine{device};

// 2. Inclusive Uniform Distribution [1, 6]
std::uniform_int_distribution<int> distribution{1, 6};

// 3. Generate Samples
for (int i = 0; i < 1000; ++i) {
    std::cout << distribution(engine) << " ";
}

std::cout << std::endl;
```

memory 1. Entropy & Engine

std::random_device: Requests non-deterministic random seed from hardware/OS.

default_random_engine: Generates pseudorandom numbers based on the seed.

casino 2. Distribution

uniform_int_distribution: Maps raw engine bits into target range [1, 6] inclusive.

3. Generate samples

distribution(engine): Invokes the engine to return a random dice roll.

Activity 0: Random Number Generation

Update code (*random_test.cpp* file available for download on class Google folder):

- 1) print random numbers generated by a **PRNG** seeded with the number 46
- 2) using **normal distribution** with mean of 4.0 and stddev 0.6

```
#include <iostream>
#include <random>

int main()
{
    std::random_device device;
    std::default_random_engine engine{device};

    std::uniform_int_distribution<int> distribution{1, 6}; //
    distribution
    for (int i = 0; i < 1000; ++i)
    {
        std::cout << distribution(engine) << " ";
    }

    std::cout << std::endl;
    return 0;
}
```

C++ References

C++ References

In C++, a **reference** is an alias to an existing object.
Once initiated, it always refer to the same memory location

Rules

- **Alias** for an object (alternative name).
- **Must be initialized** when declared.
- **Cannot be changed** to refer to another object.
- Also known as an **lvalue reference**.
- Declared by **appending &** to the type: `int& ref = existingVariable;`

Different from Python or Java references

- **Cannot be reassigned**

```
int i = 3;
int& r = i;           // r now refers to i
std::cout << r << std::endl; // writes 3
r = 4;               // a change to r is also a change to i
std::cout << i << std::endl; // writes 4
i = 5;               // a change to i is also a change to r
std::cout << r << std::endl; // writes 5
```

C++ References

- **Pass-by-value** in C++
 - function **gets a copy of the argument**
 - any changes inside function do not affect original argument
- **Pass-by-reference** in C++
 - Passing a parameter by reference means function **receives an alias**, not a copy for the original argument
- **Why?**
 - **Allows for modification** of original data
 - **Avoids copying** large data structures
 - Performance
 - Memory efficiency

```
void foo(int& x)
{
    x++;
    std::cout << x << std::endl;
}

int main()
{
    int i = 3;
    foo(i);
    std::cout << i << std::endl;
    return 0;
}
```

```
void swap(int& a, int& b)
{
    int temp = a;
    a = b;
    b = temp;
}
```

Return-by-value vs. by-reference in C++

Default: Return by Value

- **Safety First:** Guaranteed safe; avoids dangling references and undefined behavior.
- **Compiler Optimizations:** Return Value Optimization, Move Semantics, and Copy Elision (C++17) construct objects directly in caller memory with zero overhead.
- **Simpler Code:** Functions are easier to read, reason about, and chain safely.

When to Return by Reference

- **Class Getters:** Use `const T&` to expose large internal data (e.g., `std::vector`) without copying.
- **Operator Overloading:** Required for `operator[]`, `operator=`, and `operator<<` for chaining.
- **Existing Objects:** Exposing long-lived objects existing outside function scope.

CRITICAL RULE: NEVER Return Reference to Local Stack Variables!

Local objects are destroyed when stack frame is popped, resulting in dangling references and Undefined Behavior.

```
// BAD: Dangling reference to local object
int& getBad() { int local = 42; return local; } // DANGER! Destroyed on exit

// GOOD: Return by value (optimized by RVO/Copy Elision)
int getGood() { int val = 42; return val; } // Safe zero-copy return
```

Pointers

C++ Pointers

Definition

- A **pointer** is an abstraction for a memory location. It contains:
 - (1) a **memory address** - this only at runtime
 - (2) a **type** it points to (e.g. int*) - at compile time

Declared by appending * to type

```
int* x;
```

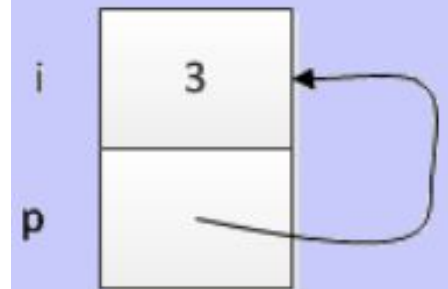
Key differences from references

- Can be null:
 - pointer can hold nullptr
- Initialization optional:
 - **do not** need to be initialized when defined
- Reassignable target:
 - after initialization, **can change** what it points to (same type)

Address-of operator &

- pointers are memory addresses
- **&** operator returns the address of a variable
- **Note:** this is different from a reference!

```
int i = 3;  
int* p = &i;
```



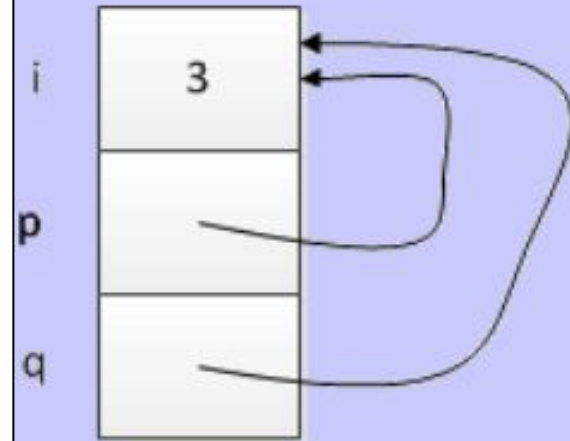
Pointers

- Pointers
 - You can assign one pointer to another
 - You can also have pointers to pointers
- Dereference operator *
 - **Precede** pointer with * to access the object a pointer points to
 - **Opposite** of & operator
 - & gets address
 - * gets the value at the address

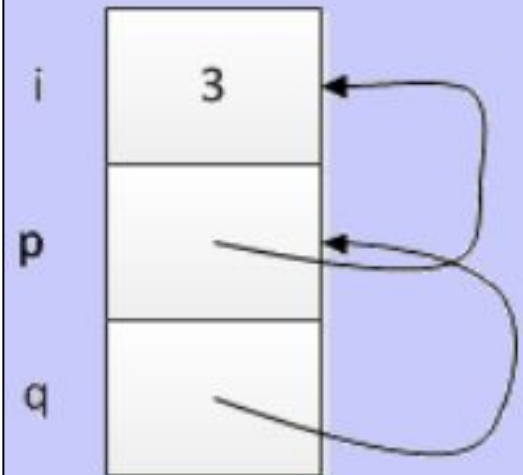
Not related to references

```
int i = 3;
int* p = &i;
std::cout << i << std::endl;
*p = 4;
std::cout << i << std::endl;
```

```
int i = 3;
int* p = &i;
int* q = p;
```



```
int i = 3;
int* p = &i;
int** q = &p;
```



C++ Pointers

Why pointers are useful

- They allow direct memory manipulation
- Enable dynamic memory allocation (e.g. new)
- Facilitate advanced data structures

```
int x = 10;  
int* p = &x;    // p holds the memory address of x  
*p = 20;        // changes x to 20 through the pointer  
  
int y = 30;  
p = &y;         // now p points to y instead of x
```

nullptr

- for tracking the absence of an object (empty or null)
- e.g. **p = nullptr**
- what happens if you try to dereference nullptr?

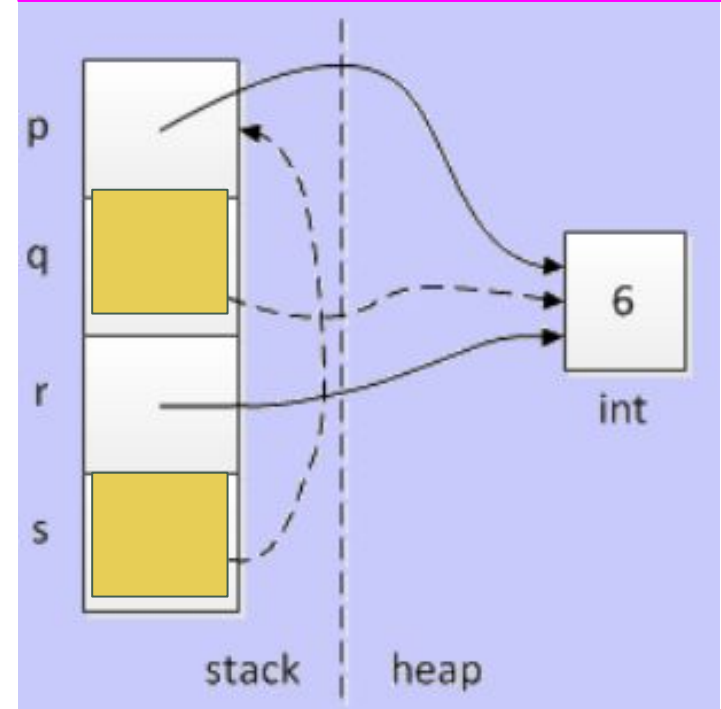
Pointers & References

- References are implicit synonyms for objects
- Pointers store location of objects

```
int* p = new int;  
*p = 3;  
  
int& q = *p;  
q = 4;  
  
std::cout << *p << std::endl;  
  
int* r = &q;  
*r = 5;  
  
std::cout << *p << std::endl;  
  
int*& s = p;  
*s = 6;  
  
std::cout << *p << std::endl;
```

read right to left:
s is a reference to int pointer

2 regions of memory
stack: function calls + local vars
heap: dynamically allocated mem



static allocation on stack
dynamic allocation on heap
dashed line = reference
solid line = pointer

Activity 1a:

```
int* a = new int;  
*a = 2;
```

```
int* b = new int;  
*b = 3;
```

```
*a = *b;
```

```
int& c = *b;  
c = 6;
```

```
std::cout << *a <<  
std::endl;
```

```
int*& d = a;  
*d = 9;
```

```
std::cout << *a <<  
std::endl;
```

```
delete a; delete b;
```

- Visualize this code the same way as the previous slide

static allocation on stack
dynamic allocation on heap
dashed line = reference
solid line = pointer

Activity 1b: Pointers & References

```
int* a = new int;  
*a = 2;
```

```
int* b = new int;  
*b = 3;
```

```
a = b; // change from 1a
```

```
int& c = *b;  
c = 6;
```

```
std::cout << *a <<  
std::endl;
```

```
int*& d = a;  
*d = 9;
```

```
std::cout << *a <<  
std::endl;
```

```
delete a; delete b;
```

- Visualize this code the same way as the previous slide

static allocation on stack
dynamic allocation on heap
dashed line = reference
solid line = pointer

Smart Pointers

Problem with Raw Pointers

Dangers of Raw Pointers

- **dereferencing of nullptr** causes undefined behavior
- may point to **invalid** locations if incorrectly cast
- **same syntax** for pointing to a single object vs an array
- will still point to deallocated objects (**dangling pointer**)

Prefer Static Allocation on Stack

- C++ auto deallocate and clean up when out of scope
- reduce risk of memory leaks and dangling pointers

Only Dynamically Allocate When Necessary

- don't know how large a data structure needs to be until runtime
- when data must outlive the function that creates it
- **new** and **delete**

Smart Pointers (Modern C++ Solution)

- **std::unique_ptr**
- exclusive ownership (most common)
- **std::share_ptr**
- shared ownership, reference counting
- **std::weak_ptr**
- non-owning "observer" pointer

Ownership

- Ownership refers to whether pointer is responsible for the lifetime of a dynamically allocated object
 - A pointer that “**owns**” an object, must ensure the object is correctly freed when pointer is destroyed (or goes out of scope).
 - Helps to prevent **memory leaks** and **double deallocation**

```
class A
{
public:
    A();
    ~A();

private:
    B* b;
    // other member variables would be here
};
```

```
A::A()
: b(new B), ... // other member variable initializers would be here
{
    // other code to be run after the initializers would be here
}
```

```
A::~~A()
{
    // necessary cleanup other than deleting the object pointed to by b
    // would be here

    delete b;
}
```

What if constructor throws exception after allocating new B, but before finishing?

deallocating the raw pointer doesn't imply deletion of the B object

Smart Pointers

- Unique ownership and **std::unique_ptr**
 - unique pointer is responsible for destroying the object it points to when being deallocated
 - guarantees automatic deletion of object **unique_ptr** points to
 - **specify type in < >**

```
#include <memory>    // We need this if we want to use smart pointers
```

```
void foo()
{
    // Creates a unique_ptr called p and points it to a newly-allocated int.
    // p holds unique ownership of that int.
    std::unique_ptr<int> p(new int);

    // We can dereference the pointer with the * operator, just as you'd
    // expect with a raw pointer.
    *p = 3;
    std::cout << *p << std::endl;

    // When p falls out of scope -- either because the function ends normally
    // or because the function threw an exception -- it will destroy the
    // int it points to automatically, so no need for delete! Similarly, if
    // we re-point p to a new integer later, or we point it to null, a
    // deletion will also be triggered.
}
```

• new syntax also

Using unique_ptr

- Rewrite class A with smart pointer
- `#include <memory>`

```
class A
{
public:
    A();
    ~A();
private:
    std::unique_ptr<B> b;
    // other member variables would go here
};

A::A()
    : b{new B}, ... // other member variable initializers would be here
{
    // other code to be run after the initializers would be here
}

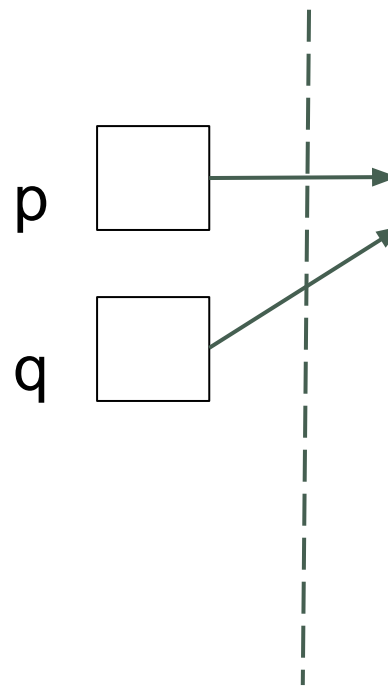
A::~~A()
{
    // necessary cleanup other than deleting the object pointed to by b
    // would go here, while there is no need to clean up anything relating to
    // b, because that will be done automatically; if there is no other
    // code needed in the destructor, we wouldn't even need to write one!
}
```

Smart Pointer

- It's **unsafe to copy unique_ptr**
 - not allowed
- What happens if we run this code w/ pass by value?

```
void foo(std::unique_ptr<int> q)
{
    *q = 4;
}

void bar()
{
    std::unique_ptr<int> p{new int};
    *p = 3;
    foo(p);
    std::cout << *p << std::endl;
}
```



Shared Ownership

- Sharing object ownership with many pointers means
 - **As long as one pointer** points to dynamically allocated object, it exists
 - Only when **last pointer is destroyed**, the object is destroyed
 - Counts shared pointers, similar to Python garbage collection
- **std::shared_ptr**
 - can be copied safely

```
void foo()
{
    std::shared_ptr<int> p1{new int};    // p1 points to a new integer
    std::shared_ptr<int> p2 = p1;        // p2 points to the same integer
    *p1 = 4;                            // 4 is stored in that integer
    std::cout << *p2 << std::endl;      // prints 4
    p1 = nullptr;                       // p1 no longer points to the integer, but p2 still does
}                                       // when the function ends, the integer is destroyed
```

Activity 2: Shared Ownership

- **std::shared_ptr**
 - Even when using shared ownership, still have memory management issues

```
#include <iostream>
#include <memory>

struct Child; // Forward declaration

struct Parent {
    std::shared_ptr<Child> child;
    ~Parent() { std::cout << "Parent destroyed\n"; }
};

struct Child {
    std::shared_ptr<Parent> parent;
    ~Child() { std::cout << "Child destroyed\n"; }
};

int main() {
    std::shared_ptr<Parent> p(new Parent());
    std::shared_ptr<Child> c(new Child());

    p->child = c;    // Parent owns Child
    c->parent = p;    // Child owns Parent

    // Neither Parent nor Child is destroyed here because
    // they keep each other alive (circular reference).
    return 0;
}
```

Activity 2: What happens to dynamically allocated parent and child at the end of main?

Why?

What's the fix?

Guidelines: Object Borrowing & Ownership

CORE PRINCIPLE

For dynamically allocated objects, not all pointers imply ownership — objects can be safely loaned (borrowed) without transferring responsibility.

A&

Use a Reference

Guaranteed Existence

Implies the object **must exist** — there is no concept of nullptr.

Const Enforcement

Use const A& to strictly enforce that the loaned/borrowed object cannot be modified.

A*

Use a Raw Pointer

Explicit Non-Ownership

By convention, passing a raw pointer assumes **no ownership** exists.

Optional Presence

Allows explicit use of nullptr when borrowing an object is optional.

std::weak_ptr

Smart Non-Ownng

Shared Ownership Pair

Holds **no ownership** of target object; safely paired with std::shared_ptr.

PRACTICAL EXAMPLE:

In tree structures, nodes use shared_ptr to children, but weak_ptr to parents to avoid memory leaks.

Activity 3:

3a) Convert raw pointers to unique pointers

3b) What happens when you try to compile? Why?

3c) Fix any necessary smart pointer types to ensure no memory leaks

3d) Convert to using only `unique_ptr` and raw ptrs for performance

```
#include <iostream>

class Node {
public:
    Node() { std::cout << "Node created\n"; }
    ~Node() { std::cout << "Node destroyed\n"; }

    // Member pointers initialized safely to nullptr
    Node* left = nullptr;
    Node* right = nullptr;
    Node* parent = nullptr;
};

int main() {
    {
        std::cout << "--- Initializing Tree Elements ---\n";
        Node* root = new Node();
        Node* leftChild = new Node();
        Node* rightChild = new Node();

        // Linking parent-to-child and child-to-parent
        root->left = leftChild;
        root->right = rightChild;

        root->left->parent = root;
        root->right->parent = root; // Complete relationship mapping

        std::cout << "--- Tearing Down Tree Elements ---\n";
        // ⚠ Manual cleanup is fragile. Deleting out-of-order causes crashes
        delete root;
        delete leftChild;
        delete rightChild;
    }

    std::cout << "End of main.\n";
    return 0;
}
```

Using raw & smart pointers together

Ownership vs Observation:

- Owners: **smart pointers control resource lifespan**
- Observers: use raw pointers when a function or component needs to read, write or inspect **w/o responsibility of destruction**

.get()

- Smart pointers provide the .get() member function to extract raw pointer address
- passes a temporary, non-owning raw pointer w/o giving up ownership

Smart Pointer Recap

- **std::unique_ptr**
 - 1 to 1 ownership between pointer and object
 - object destroyed when pointer destroyed
- **std::share_ptr**
 - many to 1 ownership between pointers and object
 - object destroyed when **last** pointer destroyed
- **std::weak_ptr**
 - no ownership, loaning
 - no cleanup when pointer destroyed
 - useful for function parameter, cyclic data structure
 -
- **.get()**
- Additional details: [Using C++11's Smart Pointers](#)

Recap

Random Number Generation in C++

True RNG

Pseudo RNG

References

Pointers

Smart pointers